

Unlock the Potential of QForms



Here's an introduction to simple yet powerful utilities that come with QForm.

It is required that forms check for valid data. Code generated forms come with built-in validation according to the constraints set on the columns in the database. Validation is also needed on other pages—for example, whether the value put in by the user in the 'Email' field is really a valid email address or not, or if 'OSFY' is an appropriate name for a person, and so on. QForms allow all this to be done quite easily using their *Form_Validate* function.

This function is created within the class definition of the QForm and is run every time the form is submitted (i.e., every time a QServerAction or QAjaxAction is executed). You can write the code for form validation within this function. Again, it would not be required to write all the validation code in the *Form_Validate* function. You can set the 'MaxLength', 'MinLength' and 'Required' properties directly in the QTextBox to get it validated.

Often, there is a need for grouping controls. For example, for a customer relationship management (CRM) Web-app, it is important to create a composite QControl to bill the customers. In such cases, it would be best to create a composite control for reuse and write the validation logic in the 'Validate' function of the composite QControl or QPanel rather than in the parent form. The strategy for how and when to use custom QControl can be found at <http://goo.gl/gFxEgP>

Custom QControls

It would be a bit difficult to explain what a custom QControl can do but the example at <http://goo.gl/XUWjx> would certainly impress and convey its scope. Typically, custom QControls are groups of other HTML controls which form a unit. This unit is highly reusable — simply use it where that group of controls is required. QControls allow writing of their own validation logic and their own layout. If you wish, a template file may be specified, which in turn will govern the layout of the control. In short, you have almost all the goodies you get with QForm, except that a QControl would not make up a page but only a set of controls.

Once defined, QControl can be used to render the entire control by adding only a few lines. Basically, a variable (a member of the instance of the QForm class) is initialised as an instance of the composite QControl, and that variable is rendered using the HTML template file. If you would rather get your hands dirty, visit <http://goo.gl/Tkp2q>.

Database-based session handling

Using a database to handle PHP sessions is not really a necessity. However, it can benefit the application as you plan to move ahead. While one server suffices to begin with, a single server, no matter what, has its limits. With the rise in popularity of your apps, sooner or later, more than one server with a